计算机组成原理 CH3 指令集与 MIPS 测试题

覆盖 PPT CH3 全知识点｜真题风格强化｜含答案解析与速查表

使用建议：先做“学生版题目”部分；做完后再看“答案解析”和最后的“扩展速查表”。本卷内容覆盖 R3-00~R3-10：ISA、CISC/RISC、MIPS 设计原则、五类指令、R/I/J、寻址方式、C 转 MIPS、过程调用、基本块、编译-汇编-链接-加载、静态/动态链接、谬误与陷阱。

注意：本卷是训练卷，题量大于正式考试，目的是把第三章常见考法一次练全。

覆盖清单

知识点	对应题目	掌握目标
ISA 与 CISC/RISC/MIPS	单选 1-3；判断 1；简答 1	理解指令集是软硬件接口，MIPS 属于 RISC
四个设计原则	单选 3-4；判断 2	简单源于规整、越小越快、好的设计需权衡、加速大概率事件
五类指令与 19 条 PPT 指令	单选 5-15；指令识别表	add/sub/lw/sw/addi/slti/andi/ori/sll/srl/and/or/nor/beq/bne/slt/j/jal/jr
R/I/J 指令格式与字段	单选 16-21；格式题	op、rs、rt、rd、shamt、funct、16 位 immediate、26 位 address
寻址方式	单选 22-27；指令识别表	立即数、寄存器、基址、PC 相对、伪直接
数组与字节编址	单选 8-9；代码填空；C 转 MIPS	word=4B，A[i] 偏移为 i*4
$zero 与立即数	单选 10-12；判断 5	没有 subi，$zero 恒为 0
分支、循环、slt+beq/bne	单选 13-15；代码填空；C 转 MIPS	if/for/while 的核心翻译
过程调用与栈	单选 28-30；代码填空；C 转 MIPS	jal/jr/$ra/$a0/$v0/$sp
高级语言翻译执行流程	单选 31-35；简答 2	编译器、汇编器、链接器、加载器，静态/动态链接
基本块与谬误陷阱	单选 36-40；判断 10-12；简答 3	Basic Block、汇编不一定最快、字节寻址陷阱、悬空指针

第一部分：单项选择题（40 题）

每题只有一个正确答案。建议限时 30 分钟完成。

1. 关于指令集体系结构 ISA 的说法，正确的是（）。

A. ISA 只属于软件，不影响硬件　B. ISA 是软件和硬件之间的接口　C. ISA 只规定晶体管如何连接　D. ISA 与编译器完全无关

2. 以下属于 RISC 指令集的是（）。

A. x86　B. MIPS　C. 复杂字符串处理指令集　D. CISC

3. 以下不属于 MIPS 指令设计基本原则的是（）。

A. 简单源于规整　B. 越小越快　C. 好的设计需要权衡　D. 坚持设计原则绝不妥协

4. “加速大概率事件”最能解释下列哪项 MIPS 设计？（）

A. 设计立即数操作和 $zero　B. 设计大量复杂内存到内存指令　C. 取消所有寄存器　D. 让每条指令长度都不固定

5. MIPS 指令操作的本质可概括为（）。

A. 让所有数据都留在外存　B. 只做浮点数运算　C. 将数从某处取出再放到某处　D. 只进行系统调用

6. MIPS 中 $t0~$t9 通常用于（）。

A. 临时值　B. 长期保存变量　C. 返回地址　D. 栈指针

7. MIPS 中 $s0~$s7 通常用于（）。

A. 临时中间值　B. 保存变量　C. 立即数常量　D. 操作码字段

8. PPT 中 32-bit data called a word，说明一个 word 是（）。

A. 1 字节　B. 2 字节　C. 4 字节　D. 8 字节

9. 若数组 A 的基地址在 $s3，按 word 访问 A[8] 的偏移量应为（）。

A. 8　B. 16　C. 32　D. 64

10. 已知 h 在 $s2，A 的基地址在 $s3，语句 A[12] = h + A[8] 的正确翻译是（）。

A. lw $t0,8($s3); add $t0,$s2,$t0; sw $t0,12($s3)　B. lw $t0,32($s3); add $t0,$s2,$t0; sw $t0,48($s3)　C. sw $t0,32($s3); add $t0,$s2,$t0; lw $t0,48($s3)　D. addi $t0,$s3,8; sw $s2,12($t0)

11. MIPS 中没有 subtract immediate 指令，若要实现 $s2 = $s1 - 1，应写成（）。

A. subi $s2,$s1,1　B. sub $s2,$s1,1　C. addi $s2,$s1,-1　D. add $s2,$s1,-1

12. 关于 $zero 的说法，正确的是（）。

A. 它恒为 0，不能被改写　B. 它用于保存返回地址　C. 它只在浮点运算中有效　D. 写入 $zero 后，下次可读出写入值

13. 若要实现 $t0 = ~$t1，最合适的是（）。

A. nor $t0,$t1,$t1　B. nor $t0,$t1,$zero　C. or $t0,$t1,$zero　D. and $t0,$t1,$zero

14. 条件“若 $s0 等于 $s1 则跳到 L1”应使用（）。

A. beq $s0,$s1,L1　B. bne $s0,$s1,L1　C. slt $s0,$s1,L1　D. j $s0,$s1,L1

15. 若要实现“若 $s1 < $s2 则跳转 L”，较标准的组合是（）。

A. beq $s1,$s2,L　B. slti $s1,$s2,L; j L　C. slt $t0,$s1,$s2; bne $t0,$zero,L　D. j $s1,$s2,L

16. 过程调用与返回通常分别使用（）。

A. j 与 beq　B. jal 与 jr $ra　C. lw 与 sw　D. addi 与 slti

17. R 型指令的字段顺序是（）。

A. op rs rt rd shamt funct　B. op rd rs rt immediate　C. op address　D. op rs rt constant

18. I 型指令中 constant or address 字段长度是（）。

A. 5 bits　B. 6 bits　C. 16 bits　D. 26 bits

19. J 型指令中 address 字段长度是（）。

A. 5 bits　B. 16 bits　C. 26 bits　D. 32 bits

20. 下列“指令-格式”匹配正确的是（）。

A. add-R，lw-I，j-J　B. add-I，lw-R，j-J　C. add-J，sw-R，beq-J　D. jr-J，jal-R，slt-I

21. 关于 R 型中的 op 和 funct，说法正确的是（）。

A. op 和 funct 都表示寄存器编号　B. op 判定大类，funct 判定具体 R 型操作　C. funct 只用于 I 型　D. op 只用于 J 型

22. 以下不属于 MIPS 常见寻址方式的是（）。

A. 立即数寻址　B. 相对基址变址寻址　C. PC 相对寻址　D. 基址寻址

23. addi $s0,$s1,10 的主要寻址方式是（）。

A. 立即数寻址　B. 基址寻址　C. PC 相对寻址　D. 伪直接寻址

24. add $s0,$s1,$s2 的主要寻址方式是（）。

A. 立即数寻址　B. 寄存器寻址　C. 基址寻址　D. 伪直接寻址

25. lw $s0,32($s4) 的有效地址是（）。

A. $s0 + 32　B. $s4 + 32　C. PC + 32　D. 32

26. beq $s0,$s1,L1 采用的目标地址形成方式是（）。

A. PC 相对寻址　B. 基址寻址　C. 伪直接寻址　D. 寄存器间接寻址

27. j L1 与 jal Procedure 采用的目标地址形成方式通常称为（）。

A. 立即数寻址　B. PC 相对寻址　C. 基址寻址　D. 伪直接寻址

28. jr $ra 是 R 型，主要因为（）。

A. 它的跳转目标来自寄存器　B. 它含 26 位 address　C. 它一定访问内存　D. 它是立即数指令

29. 函数调用时，$ra 的作用是（）。

A. 保存返回地址　B. 保存第一个参数　C. 保存函数返回值　D. 保存栈顶地址

30. 函数调用时，$sp 的作用最准确的是（）。

A. 保存立即数　B. 指向栈顶，用于保存现场　C. 保存机器码　D. 保存操作码

31. 下列工具链顺序正确的是（）。

A. C 程序->链接器->汇编器->编译器->加载器　B. C 程序->汇编器->编译器->链接器->加载器　C. C 程序->编译器->汇编器->链接器->加载器　D. C 程序->加载器->链接器->汇编器->编译器

32. 负责把目标文件和库例程合成为可执行文件，并处理外部符号/重定位的是（）。

A. 编译器　B. 链接器　C. 加载器　D. 文本编辑器

33. 负责将可执行文件放入内存并开始运行的是（）。

A. 汇编器　B. 编译器　C. 加载器　D. 预处理器

34. 动态链接库的特点是（）。

A. 程序执行过程中才被链接/加载　B. 一定把全部库函数复制进可执行文件　C. 不能更新库函数　D. 只能用于很小的程序

35. 以下关于基本块 Basic Block 的说法，错误的是（）。

A. 是连续一段指令　B. 内部不含任何分支　C. 可由编译器划分　D. 仅调用自身的嵌套递归函数可以认为是基本块

36. MIPS 指令以二进制编码并存储在内存中，这体现的是（）。

A. 存储程序概念　B. 只能用硬件写程序　C. 程序无法操作程序　D. 指令和数据不能同存在内存

37. 二进制兼容的含义更接近（）。

A. 已编译程序可在兼容 ISA 的机器上运行　B. 所有机器必须采用同一时钟频率　C. 所有语言都不需要编译　D. 指令集永远不能添加新指令

38. 关于“汇编语言一定比高级语言快”，正确判断是（）。

A. 必然正确　B. 通常错误，现代编译器优化很强　C. 只在 MIPS 中正确　D. 与编译器无关

39. 字节寻址机器中，连续两个 32-bit word 的地址通常相差（）。

A. 1　B. 2　C. 4　D. 32

40. 下列 C 代码中，最可能造成悬空指针问题的是（）。

A. 返回局部变量的地址　B. 使用 addi 加立即数　C. 使用数组下标乘 4　D. 调用 printf

第二部分：判断改错题（12 题）

判断正误；错误的请写出正确说法。

1. （ ）MIPS 属于 RISC 指令集，x86 通常归入 CISC 指令集。

2. （ ）MIPS 的设计原则之一是“坚持所有原则绝不权衡”。

3. （ ）R 型指令中的每个字段长度都是 5 bits。

4. （ ）只有 lw 和 sw 这类加载/存储指令直接访问数据存储器；算术/逻辑指令主要处理寄存器中的值。

5. （ ）add $zero,$t2,$s1 可以改变 $zero 的值。

6. （ ）`lw $t0, 0($s1)` 属于基址寻址，地址由 $s1 和偏移量 0 相加得到。

7. （ ）beq/bne 是 I 型指令，但它们不是立即数寻址，而是 PC 相对寻址。

8. （ ）j 和 jal 是 J 型指令，属于伪直接寻址。

9. （ ）jr $ra 是 J 型指令，因为它也是跳转。

10. （ ）动态链接有利于库函数更新，也能让多个程序共享库。

11. （ ）一个基本块内部可以包含任意多条分支指令，只要最后能返回。

12. （ ）如果函数返回局部变量地址，调用者继续使用该地址可能出现错误。

第三部分：指令识别表（19 条 PPT 指令全覆盖）

请填写 R/I/J、寻址方式，并写出指令作用。

序号	指令	R/I/J	寻址方式	作用
1	add $s0,$s1,$s2			
2	sub $s0,$s1,$s2			
3	lw $s0,32($s4)			
4	sw $s0,48($s4)			
5	addi $s0,$s1,-1			
6	slti $s0,$s1,-1			
7	andi $s0,$s1,10			
8	ori $s0,$s1,10			
9	sll $s0,$s1,4			
10	srl $s0,$s1,4			
11	and $s0,$s1,$s2			
12	or $s0,$s1,$s2			
13	nor $s0,$s1,$s2			
14	beq $s0,$s1,L1			
15	bne $s0,$s1,L1			
16	slt $s0,$s1,$s2			
17	j L1			
18	jal Procedure			
19	jr $ra			

第四部分：代码填空与汇编阅读

1. 数组访问。已知 h 在 $s2，A 基地址在 $s3，请补全 A[12] = h + A[8]。

```
____ $t0, 32($s3)       # $t0 = A[8]
____ $t0, $s2, $t0      # $t0 = h + A[8]
```

```
____ $t0, 48($s3)      # A[12] = $t0
```

2. if 判断。已知 a=$s0，b=$s1，若 a<b 则跳到 Less，请补全。

```
____ $t0, $s0, $s1     # $t0 = (a < b)
____ $t0, $zero, Less  # if ($t0 != 0) goto Less
```

3. while 循环。已知 i=$s0，n=$s1，sum=$s2，A 基地址=$s3。补全 while(i<n){sum+=A[i]; i++;}。

```
Loop:
    ____ $t0, $s0, $s1      # i < n ?
    ____ $t0, $zero, Body
    ____ Exit
Body:
    ____ $t1, $s0, 2        # i * 4
    ____ $t1, $s3, $t1      # &A[i]
    ____ $t2, 0($t1)        # A[i]
    ____ $s2, $s2, $t2      # sum += A[i]
    ____ $s0, $s0, 1        # i++
    ____ Loop
Exit:
```

4. 过程调用。补全函数 add2(x,y)：返回 x+y+2。约定 x 在 $a0，y 在 $a1，返回值放 $v0。

```
add2:
    ____ $v0, $a0, $a1
    ____ $v0, $v0, 2
    ____ $ra
```

5. 栈保存现场。函数 f 调用 g 前需要保存 $ra 和 $a0，请补全。

```
f:
    ____ $sp, $sp, -8
    ____ $ra, 4($sp)
    ____ $a0, 0($sp)
    jal g
    ____ $a0, 0($sp)
    ____ $ra, 4($sp)
    ____ $sp, $sp, 8
    jr $ra
```

第五部分：C 代码翻译成 MIPS

要求：使用 PPT 出现过的核心指令；如果使用临时寄存器，请优先使用 $t0,$t1,$t2。

1. 已知 f=$s0, g=$s1, h=$s2, i=$s3, j=$s4，翻译：f = (g + h) - (i + j);

答：

2. 已知 a=$s0, b=$s1, c=$s2，翻译：a = (b & c) | ~(b | c);

答:

3. 已知 A 基地址=$s4, i=$s3, k=$s5，翻译：A[i] = (A[i] + k) | 10;

答:

4. 已知 x=$s0, y=$s1，翻译：if (x != y) x = x - 1; else y = y + 1;

答:

第六部分：格式、寻址与机器码概念

1. 对下列指令分别指出格式、主要字段含义与寻址方式。

```
add $t0,$t1,$t2
lw  $t0,32($s3)
beq $s0,$s1,L1
```

```
j   L1
jal Procedure
jr  $ra
```

2. 说明 R 型中 op 与 funct 的分工。

答：

3. 为什么 I 型不等于“立即数寻址”？请至少举 3 个 I 型指令说明。

答：

第七部分：高级语言如何被翻译并执行

1. 填出流程：C program -> _____ -> assembly program -> _____ -> object file -> _____ -> executable file -> _____ -> memory。

2. 说明编译器、汇编器、链接器、加载器分别做什么。

3. 比较静态链接和动态链接库；说明为什么动态链接有利于函数库更新、程序只用到部分库函数、运行较大程序。

第八部分：综合分析题（真题风格）

1. 阅读下列代码，划分基本块，并说明每个基本块的入口和出口。

```
Loop:
    slt  $t0,$s0,$s1
    beq  $t0,$zero,Exit
    lw   $t1,0($s2)
    add  $s3,$s3,$t1
    addi $s0,$s0,1
    j    Loop
Exit:
    jr   $ra
```

2. 结合 MIPS 设计原则说明：为什么 MIPS 采用较规整的 R/I/J 格式，而不是给每一种高级语言语句都设计一条复杂指令？

3. 判断并解释：手写汇编语言一定能获得最高性能；强指令一定更快；成功的指令集不需要改变。

答案解析

第一部分：单选题答案

1.B 2.B 3.D 4.A 5.C 6.A 7.B 8.C 9.C 10.B 11.C 12.A 13.B 14.A 15.C 16.B 17.A 18.C 19.C 20.A 21.B 22.B 23.A 24.B 25.B 26.A 27.D 28.A 29.A 30.B 31.C 32.B 33.C 34.A 35.D 36.A 37.A 38.B 39.C 40.A

重点解析：

- 3：MIPS 的设计原则不是“永不妥协”，而是好的设计需要权衡。

- 9-10：内存按字节编址，word=4B，因此 A[8] 偏移 32，A[12] 偏移 48。

- 11：没有 subi，减立即数用 addi 加负数。

- 13：按位取反可用 nor x, y, $zero，因为 y | 0 = y，再取反。

- 15：小于判断通常先 slt 置 0/1，再用 bne/beq 分支。

- 17-21：R 型字段为 op rs rt rd shamt funct；I 型最后 16 位；J 型 address 26 位。

- 22-27：I 型中 addi/slti/andi/ori 是立即数寻址，lw/sw 是基址寻址，beq/bne 是 PC 相对寻址。

- 28：jr $ra 虽然是跳转，但目标来自寄存器，所以归入 R 型。

- 31-34：工具链顺序是编译、汇编、链接、加载；链接器负责外部符号解析和地址重定位。

- 35：基本块内部不能含分支；递归调用不自动构成基本块。

第二部分：判断改错答案

1. 对。

2. 错。应为：好的设计需要权衡。

3. 错。R 型字段长度是 6/5/5/5/5/6，不是全 5 位。

4. 对。

5. 错。$zero 恒为 0，不能被改写。

6. 对。

7. 对。

8. 对。

9. 错。jr $ra 是 R 型，目标地址来自寄存器。

10. 对。

11. 错。基本块是连续一段指令，内部不含分支。

12. 对。

第三部分：指令识别表答案

指令	R/I/J	寻址方式	作用
add $s0,$s1,$s2	R	寄存器寻址	$s0=$s1+$s2

sub $s0,$s1,$s2	R	寄存器寻址	$s0=$s1-$s2
lw $s0,32($s4)	I	基址寻址	$s0=Memory[$s4+32]
sw $s0,48($s4)	I	基址寻址	Memory[$s4+48]=$s0
addi $s0,$s1,-1	I	立即数寻址	$s0=$s1-1
slti $s0,$s1,-1	I	立即数寻址	$s0=($s1<-1)?1:0
andi $s0,$s1,10	I	立即数寻址	$s0=$s1&10
ori $s0,$s1,10	I	立即数寻址	$s0=$s1|10
sll $s0,$s1,4	R	寄存器寻址/移位量 shamt	$s0=$s1<<4
srl $s0,$s1,4	R	寄存器寻址/移位量 shamt	$s0=$s1>>4
and $s0,$s1,$s2	R	寄存器寻址	$s0=$s1&$s2
or $s0,$s1,$s2	R	寄存器寻址	$s0=$s1|$s2
nor $s0,$s1,$s2	R	寄存器寻址	$s0=~($s1|$s2)
beq $s0,$s1,L1	I	PC 相对寻址	if($s0==$s1) goto L1
bne $s0,$s1,L1	I	PC 相对寻址	if($s0!=$s1) goto L1
slt $s0,$s1,$s2	R	寄存器寻址	$s0=($s1<$s2)?1:0
j L1	J	伪直接寻址	goto L1
jal Procedure	J	伪直接寻址	call Procedure, $ra=return address
jr $ra	R	寄存器寻址	return to address in $ra

第四部分：代码填空答案

1) lw / add / sw

2) slt / bne

3)

```
Loop:
    slt  $t0, $s0, $s1
    bne  $t0, $zero, Body
    j    Exit
Body:
    sll  $t1, $s0, 2
    add  $t1, $s3, $t1
    lw   $t2, 0($t1)
    add  $s2, $s2, $t2
    addi $s0, $s0, 1
    j    Loop
Exit:
```

4)

```
add2:
    add  $v0, $a0, $a1
    addi $v0, $v0, 2
    jr   $ra
```

5)

```
f:
    addi $sp, $sp, -8
    sw   $ra, 4($sp)
    sw   $a0, 0($sp)
    jal  g
    lw   $a0, 0($sp)
    lw   $ra, 4($sp)
```

```
    addi $sp, $sp, 8
    jr   $ra
```

第五部分：C 转 MIPS 参考答案

1. f = (g + h) - (i + j);

```
add $t0, $s1, $s2     # t0 = g + h
add $t1, $s3, $s4     # t1 = i + j
sub $s0, $t0, $t1     # f = t0 - t1
```

2. a = (b & c) | ~(b | c);

```
and $t0, $s1, $s2     # b & c
or  $t1, $s1, $s2     # b | c
nor $t1, $t1, $zero   # ~(b | c)
or  $s0, $t0, $t1     # result
```

3. A[i] = (A[i] + k) | 10;

```
sll  $t0, $s3, 2      # offset = i * 4
add  $t0, $s4, $t0    # address of A[i]
lw   $t1, 0($t0)      # A[i]
add  $t1, $t1, $s5    # A[i] + k
ori  $t1, $t1, 10     # | 10
sw   $t1, 0($t0)      # store back
```

4. if (x != y) x = x - 1; else y = y + 1;

```
bne  $s0, $s1, Then
addi $s1, $s1, 1      # else: y++
j    Exit
Then:
addi $s0, $s0, -1     # x--
Exit:
```

第六部分：格式、寻址与机器码概念答案

1. add 为 R 型，寄存器寻址；lw 为 I 型，基址寻址；beq 为 I 型，PC 相对寻址；j/jal 为 J 型，伪直接寻址；jr 为 R 型，跳转目标在寄存器 $ra 中。

2. R 型中 op 字段标识这是 R 类/特殊类操作，具体是 add、sub、and、or、slt 等由 funct 字段决定；rs、rt 提供源寄存器，rd 为目的寄存器，shamt 用于 sll/srl。

3. I 型只是机器码格式，最后 16 位字段可作常数、访存偏移或分支偏移。例如 addi 是立即数寻址，lw/sw 是基址寻址，beq/bne 是 PC 相对寻址。

第七部分：高级语言翻译执行答案

1. C program -> Compiler -> assembly program -> Assembler -> object file -> Linker -> executable file -> Loader -> memory。

2. 编译器：高级语言到汇编/目标层次；汇编器：汇编到机器码目标文件；链接器：连接目标文件与库例程、解析外部符号并重定位，生成可执行文件；加载器：把可执行文件装入内存并启动。

3. 静态链接在生成可执行文件时完成，库代码/库例程更早并入程序；动态链接在程序执行过程中链接/加载。动态链接便于库函数更新，多个程序共享同一库，程序只用到部分库函数时不必全部复制进可执行文件；也有利于节省内存和支持较大程序运行。

第八部分：综合分析题参考答案

1. 基本块划分可为：B1=Loop 到 beq（入口 Loop，出口 beq 分到 Body/Exit）；B2=lw/add/addi/j（顺序执行到 j Loop）；B3=Exit: jr $ra。基本块内部不含分支，分支/跳转通常结束一个基本块。

2. 因为 MIPS 遵循简单源于规整、越小越快、好的设计需要权衡、加速大概率事件。规整格式便于硬件译码、流水线和编译器生成代码；为每种高级语句设计复杂指令会增加硬件复杂性，未必带来总体性能提升。

3. 三个说法都不绝对。手写汇编不一定最快，现代编译器优化能力强；强指令不一定更快，复杂指令可能增加解码和实现代价；成功指令集通常要保持二进制兼容，但仍会增加扩展指令，兼容性与演进需要权衡。

附录 A：扩展速查表（由原“指令”表扩充）

这张表用于考前快速复盘：五类功能、PPT 指令、R/I/J、寻址方式、格式例子、作用和高级语言含义。

指令	R/I/J	寻址方式	格式例子	作用/高级语言含义
add	R	寄存器寻址	add $s0,$s1,$s2	$s0=$s1+$s2
sub	R	寄存器寻址	sub $s0,$s1,$s2	$s0=$s1-$s2
lw	I	基址寻址	lw $s0,32($s4)	$s0=Memory[$s4+32]
sw	I	基址寻址	sw $s0,48($s4)	Memory[$s4+48]=$s0
addi	I	立即数寻址	addi $s0,$s1,-1	$s0=$s1-1
slti	I	立即数寻址	slti $s0,$s1,-1	$s0=($s1<-1)?1:0
andi	I	立即数寻址	andi $s0,$s1,10	$s0=$s1&10
ori	I	立即数寻址	ori $s0,$s1,10	$s0=$s1|10
sll	R	寄存器寻址/移位量 shamt	sll $s0,$s1,4	$s0=$s1<<4
srl	R	寄存器寻址/移位量 shamt	srl $s0,$s1,4	$s0=$s1>>4
and	R	寄存器寻址	and $s0,$s1,$s2	$s0=$s1&$s2
or	R	寄存器寻址	or $s0,$s1,$s2	$s0=$s1|$s2
nor	R	寄存器寻址	nor $s0,$s1,$s2	$s0=~($s1|$s2)
beq	I	PC 相对寻址	beq $s0,$s1,L1	if($s0==$s1) goto L1
bne	I	PC 相对寻址	bne $s0,$s1,L1	if($s0!=$s1) goto L1
slt	R	寄存器寻址	slt $s0,$s1,$s2	$s0=($s1<$s2)?1:0
j	J	伪直接寻址	j L1	goto L1
jal	J	伪直接寻址	jal Procedure	call Procedure, $ra=return address
jr	R	寄存器寻址	jr $ra	return to address in $ra

附录 B：五种寻址方式最小口诀

常数直接出现 -> 立即数寻址；全是寄存器 -> 寄存器寻址；offset(base) -> 基址寻址；beq/bne -> PC 相对寻址；j/jal -> 伪直接寻址。

特别注意：I 型不等于立即数寻址。I 型可能是立即数寻址、基址寻址或 PC 相对寻址。